2
Data Structures

Data structures are the heart of any sophisticated program. Selecting the right data structure can make an enormous difference in the complexity of the resulting implementation. Pick the right data representation, and your task will be easy to program. Pick the wrong representation, and you may spend enormous amounts of time and code covering up for your lousy initial decision.

In this chapter, we will review the fundamental data structures which every programmer should be familiar with. We will motivate this discussion with an example program based on a children's card game. Many classic programming problems are based on games. After all, who makes it through their introductory programming courses without moving the towers of Hanoi, taking the knight's tour, or solving the eight queens problem?

2.1 Elementary Data Structures

Here we present the abstract operations of the most important data structures: stacks, queues, dictionaries, priority queues, and sets. We also describe the simplest way to implement these structures from scratch.

Be aware, however, that modern object-oriented languages like C++ and Java come with standard libraries of fundamental data structures. These will be briefly described in Section 2.2. It is worth every programmer's time to get acquainted with your libraries instead of repeatedly reinventing the wheel. Once you do, you can read this section with an eye for what each data structure is good for instead of the details of how it should be implemented.

2.1.1 Stacks

Stacks and queues are containers where items are retrieved according to the order of insertion, independent of content. *Stacks* maintain *last-in, first-out* order. The abstract operations on a stack include –

- *Push(x,s)* — Insert item x at the top of stack s.
- *Pop(s)* — Return (and remove) the top item of stack s.
- *Initialize(s)* — Create an empty stack.
- *Full(s), Empty(s)* — Test whether the stack can accept more pushes or pops, respectively.

Note that there is no element search operation defined on standard stacks and queues.

Defining these abstract operations enables us build a stack module to use and reuse without knowing the details of the implementation. The easiest implementation uses an array with an index variable to represent the top of the stack. An alternative implementation, using linked lists, is better because it can't overflow.

Stacks naturally model piles of objects, such as dinner plates. After a new plate is washed, it is pushed on the top of the stack. When someone is hungry, a clean plate is popped off the top. A stack is an appropriate data structure for this task since the plates don't care which one is used next. Thus one important application of stacks is whenever order *doesn't* matter, because stacks are particularly simple containers to implement.

Stack order is important in processing any properly nested structure. This includes parenthesized formulas (push on a "(", pop on ")"), recursive program calls (push on a procedure entry, pop on a procedure exit), and depth-first traversals of graphs (push on discovering a vertex, pop on leaving it for the last time).

2.1.2 Queues

Queues maintain *first-in, first-out* order. This appears fairer than last-in, first-out, which is why lines at stores are organized as queues instead of stacks. Decks of playing cards can be modeled by queues, since we deal the cards off the top of the deck and add them back in at the bottom. FIFO queues will be used in implementing breadth-first search in graphs in Chapter 9.

The abstract operations on a queue include —

- *Enqueue(x,q)* — Insert item x at the back of queue q.
- *Dequeue(q)* — Return (and remove) the front item from queue q
- *Initialize(q), Full(q), Empty(q)* — Analogous to these operation on stacks.

Queues are more difficult to implement than stacks, because action happens at both ends. The *simplest* implementation uses an array, inserting new elements at one end and *moving* all remaining elements to fill the empty space created on each dequeue.

However, it is very wasteful to move all the elements on each dequeue. How can we do better? We can maintain indices to the first (head) and last (tail) elements in the

array/queue and do all operations locally. There is no reason why we must explicitly clear previously used cells, although we leave a trail of garbage behind the previously dequeued items.

Circular queues let us reuse this empty space. Note that the pointer to the front of the list is always *behind* the back pointer! When the queue is full, the two indices will point to neighboring or identical elements. There are several possible ways to adjust the indices for circular queues. *All are tricky!* The easiest solution distinguishes full from empty by maintaining a count of how many elements exist in the queue:

```
typedef struct {
        int q[QUEUESIZE+1];             /* body of queue */
        int first;                      /* position of first element */
        int last;                       /* position of last element */
        int count;                      /* number of queue elements */
} queue;

init_queue(queue *q)
{
        q->first = 0;
        q->last = QUEUESIZE-1;
        q->count = 0;
}

enqueue(queue *q, int x)
{
        if (q->count >= QUEUESIZE)
                printf("Warning: queue overflow enqueue x=%d\n",x);
        else {
                q->last = (q->last+1) % QUEUESIZE;
                q->q[ q->last ] = x;
                q->count = q->count + 1;
        }
}

int dequeue(queue *q)
{
        int x;

        if (q->count <= 0) printf("Warning: empty queue dequeue.\n");
        else {
                x = q->q[ q->first ];
                q->first = (q->first+1) % QUEUESIZE;
                q->count = q->count - 1;
        }
```

```
        return(x);
}

int empty(queue *q)
{
        if (q->count <= 0) return (TRUE);
        else return (FALSE);
}
```

Queues are one of the few data structures which are easier to program using linked lists than arrays, because they eliminate the need to test for the wrap-around condition.

2.1.3 Dictionaries

Dictionaries permit content-based retrieval, unlike the position-based retrieval of stacks and queues. They support three primary operations –

- *Insert(x,d)* — Insert item x into dictionary d.
- *Delete(x,d)* — Remove item x (or the item pointed to by x) from dictionary d.
- *Search(k,d)* — Return an item with key k if one exists in dictionary d.

A data structures course may well present a dozen different ways to implement dictionaries, including sorted/unsorted linked lists, sorted/unsorted arrays, and a forest full of random, splay, AVL, and red-black trees – not to mention all the variations on hashing.

The primary issue in algorithm analysis is performance, namely, achieving the best possible trade-off between the costs of these three operations. But what we usually want in practice is the simplest way to get the job done under the given time constraints. The right answer depends on how much the contents of your dictionary change over the course of execution:

- *Static Dictionaries* — These structures get built once and never change. Thus they need to support search, but not insertion on deletion.

  The right answer for static dictionaries is typically an array. The only real question is whether to keep it sorted, in order to use binary search to provide fast membership queries. Unless you have tight time constraints, it probably isn't worth using binary search until $n > 100$ or so. You might even get away with sequential search to $n = 1,000$ or more, provided you will not be doing too many searches.

  Sorting algorithms and binary search always prove harder to debug than they should. Library sort/search routines are available for C, C++, and Java, and will be presented in Chapter 4.

- *Semi-dynamic Dictionaries* — These structures support insertion and search queries, but not deletion. If we know an upper bound on the number of elements to be inserted we can use an array, but otherwise we must use a linked structure.

Hash tables are excellent dictionary data structures, particularly if deletion need not be supported. The idea is to apply a function to the search key so we can determine *where* the item will appear in an array without looking at the other items. To make the table of reasonable size, we must allow for *collisions*, two distinct keys mapped to the same location.

The two components to hashing are (1) defining a hash function to map keys to integers in a certain range, and (2) setting up an array as big as this range, so that the hash function value can specify an index.

The basic hash function converts the key to an integer, and takes the value of this integer mod the size of the hash table. Selecting a table size to be a prime number (or at least avoiding obvious composite numbers like 1,000) is helpful to avoid trouble. Strings can be converted to integers by using the letters to form a base "alphabet-size" number system. To convert "steve" to a number, observe that e is the 5th letter of the alphabet, s is the 19th letter, t is the 20th letter, and v is the 22nd letter. Thus "steve" $\Rightarrow 26^4 \times 19 + 26^3 \times 20 + 26^2 \times 5 + 26^1 \times 22 + 26^0 \times 5 = 9,038,021$. The first, last, or middle ten characters or so will likely suffice for a good index. Tricks on how to do the modular arithmetic efficiently will be discussed in Chapter 7.

The absence of deletion makes *open addressing* a nice, easy way to resolve collisions. In open addressing, we use a simple rule to decide where to put a new item when the desired space is already occupied. Suppose we always put it in the next unoccupied cell. On searching for a given item, we go to the intended location and search sequentially. If we find an empty cell before we find the item, it does not exist anywhere in the table.

Deletion in an open addressing scheme is very ugly, since removing one element can break a chain of insertions, making some elements inaccessible. The key to efficiency is using a large-enough table that contains many holes. Don't be cheap in selecting your table size or else you will pay the price later.

- *Fully Dynamic Dictionaries* — Hash tables are great for fully dynamic dictionaries as well, provided we use *chaining* as the collision resolution mechanism. Here we associate a linked list with each table location, so insertion, deletion, and query reduce to the same problem in linked lists. If the hash function does a good job the m keys will be distributed uniformly in a table of size n, so each list will be short enough to search quickly.

2.1.4 Priority Queues

Priority queues are data structures on sets of items supporting three operations –

- *Insert(x,p)* — Insert item x into priority queue p.
- *Maximum(p)* — Return the item with the largest key in priority queue p.
- *ExtractMax(p)* — Return and remove the item with the largest key in p.

Priority queues are used to maintain schedules and calendars. They govern who goes next in simulations of airports, parking lots, and the like, whenever we need to schedule events according to a clock. In a human life simulation, it may be most convenient to determine when someone will die immediately after they are born. We can then stick this date in a priority queue so as to be reminded when to bury them.

Priority queues are used to schedule events in the sweep-line algorithms common to computational geometry. Typically, we use the priority queue to store the points we have not yet encountered, ordered by x-coordinate, and push the line forward one step at a time.

The most famous implementation of priority queues is the binary heap, which can be efficiently maintained in either a top-down or bottom-up manner. Heaps are very slick and efficient, but may be tricky to get right under time pressure. Far simpler is maintaining a sorted array, particularly if you will not be performing too many insertions.

2.1.5 Sets

Sets (or more strictly speaking *subsets*) are unordered collections of elements drawn from a given universal set U. Set data structures get distinguished from dictionaries because there is at least an implicit need to encode which elements from U are *not* in the given subset.

The basic operations on subsets are —

- *Member(x,S)* — Is an item x an element of subset S?
- *Union(A,B)* — Construct subset $A \cup B$ of all elements in subset A or in subset B.
- *Intersection(A,B)* — Construct subset $A \cap B$ of all elements in subset A and in subset B.
- *Insert(x,S), Delete(x,S)* — Insert/delete element x into/from subset S.

For sets of a large or unbounded universe, the obvious solution is representing a subset using a dictionary. Using sorted dictionaries makes union and intersection operations much easier, basically reducing the problem to merging two sorted dictionaries. An element is in the union if it appears at least once in the merged list, and in the intersection if it appears exactly twice.

For sets drawn from small, unchanging universes, bit vectors provide a convenient representation. An n-bit vector or array can represent any subset S from an n-element universe. Bit i will be 1 iff $i \in S$. Element insertion and deletion operations simply flip the appropriate bit. Intersection and union are done by "and-ing" or "or-ing" the corresponding bits together. Since only one bit is used per element, bit vectors can be space efficient for surprisingly large values of $|U|$. For example, an array of 1,000 standard four-byte integers can represent any subset on 32,000 elements.